Universidad Tecnológica del Perú

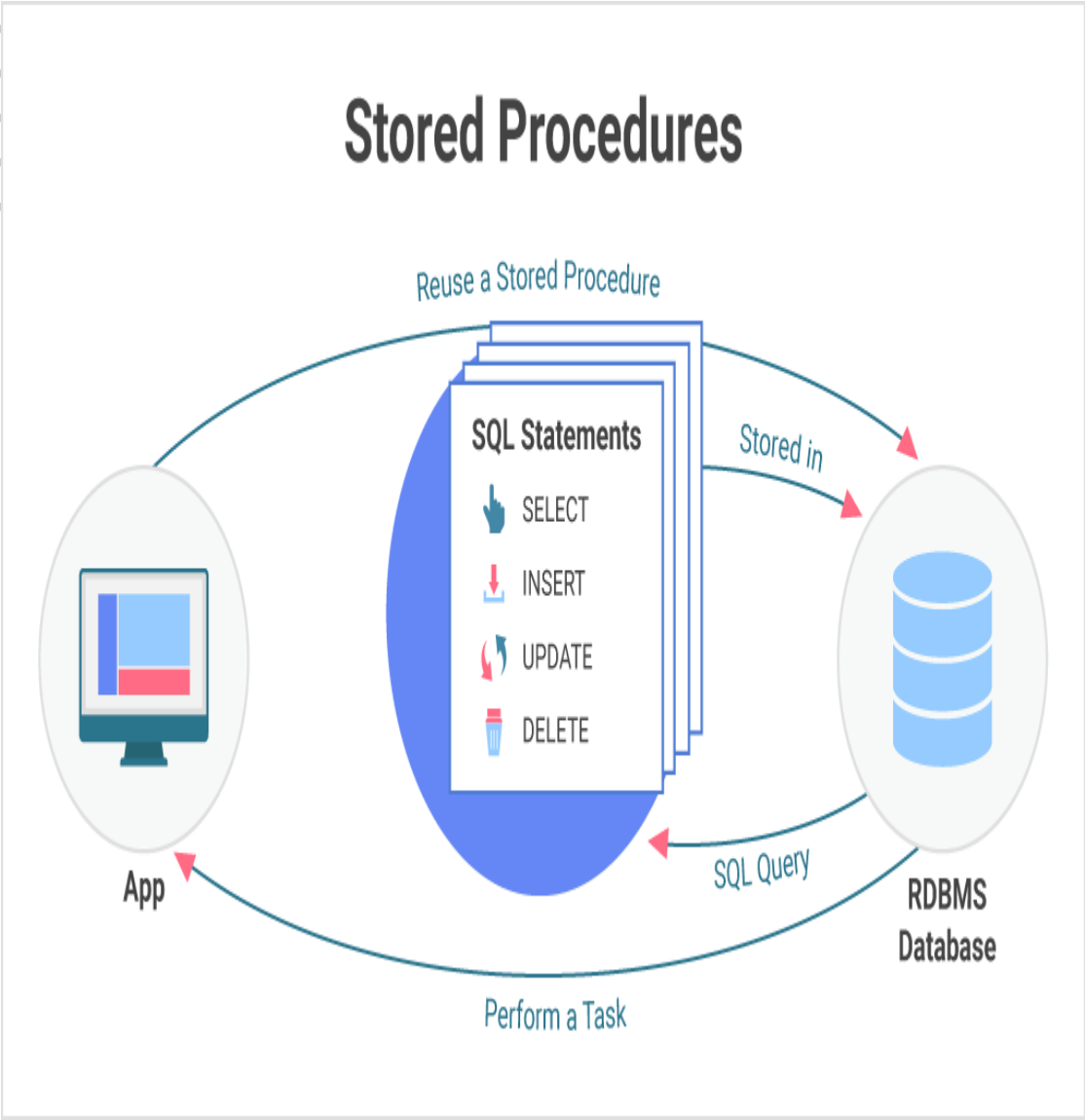
BASE DE DATOS

Mg.Ing. Yuri A. Pomachagua Basualdo

Daisuke Kishimoto

2025

Desaprende lo que te limita



Universidad Tecnológica del Perú

Transact SQL Funciones y Procedimientos Almacenados

Desaprende lo que te limita

Ing Yuri A. Pomachagua Basualdo

Agenda

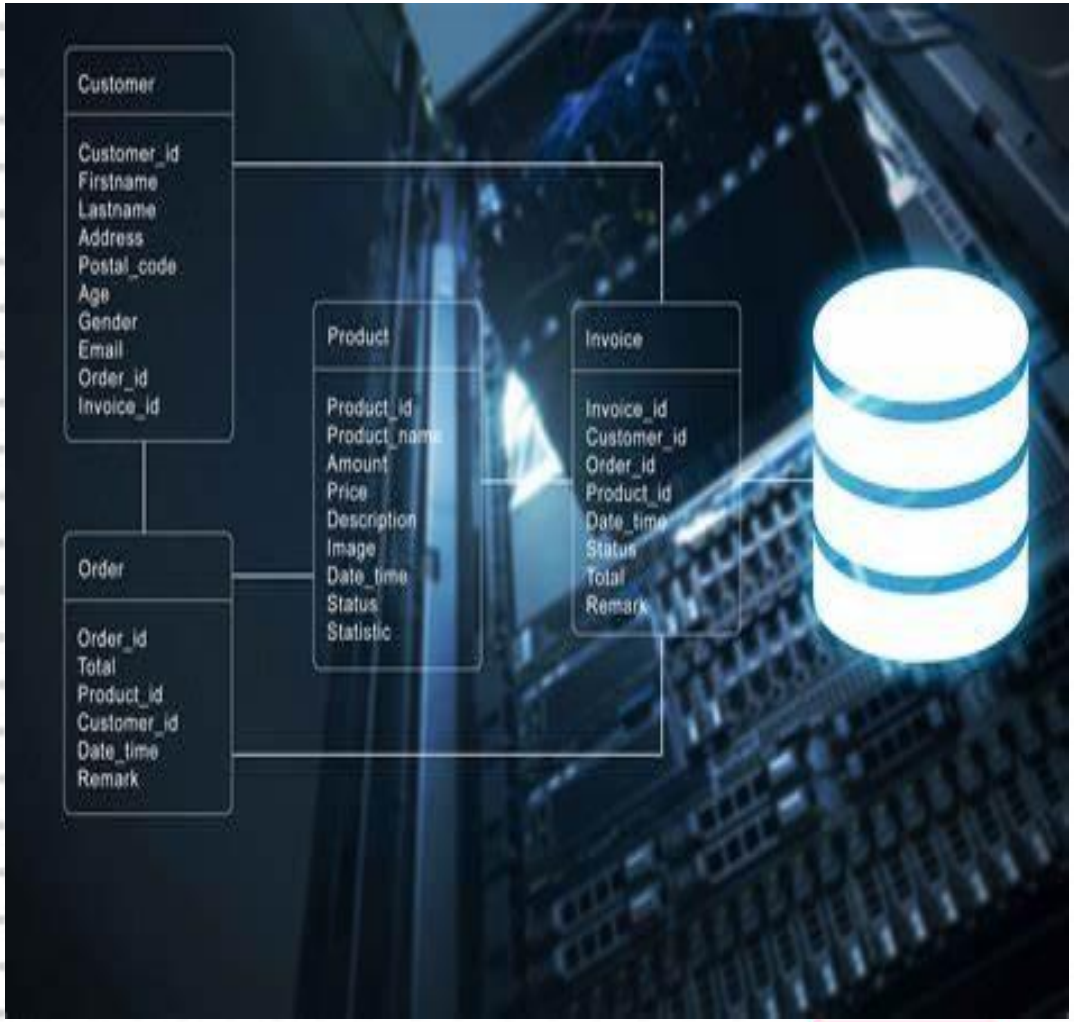
- Transact SQL
- Creación de funciones y procedimientos almacenados.
- Manejo de variables y condicional IF
- Programación de cursores en Transact SQL

VectorStock® VectorStock.com/30487804

Logro

Al finalizar la sesión el estudiante conoce las funciones y procedimientos almacenados, para aplicar en las consultas de la base de datos.

Universidad Tecnológica del Perú



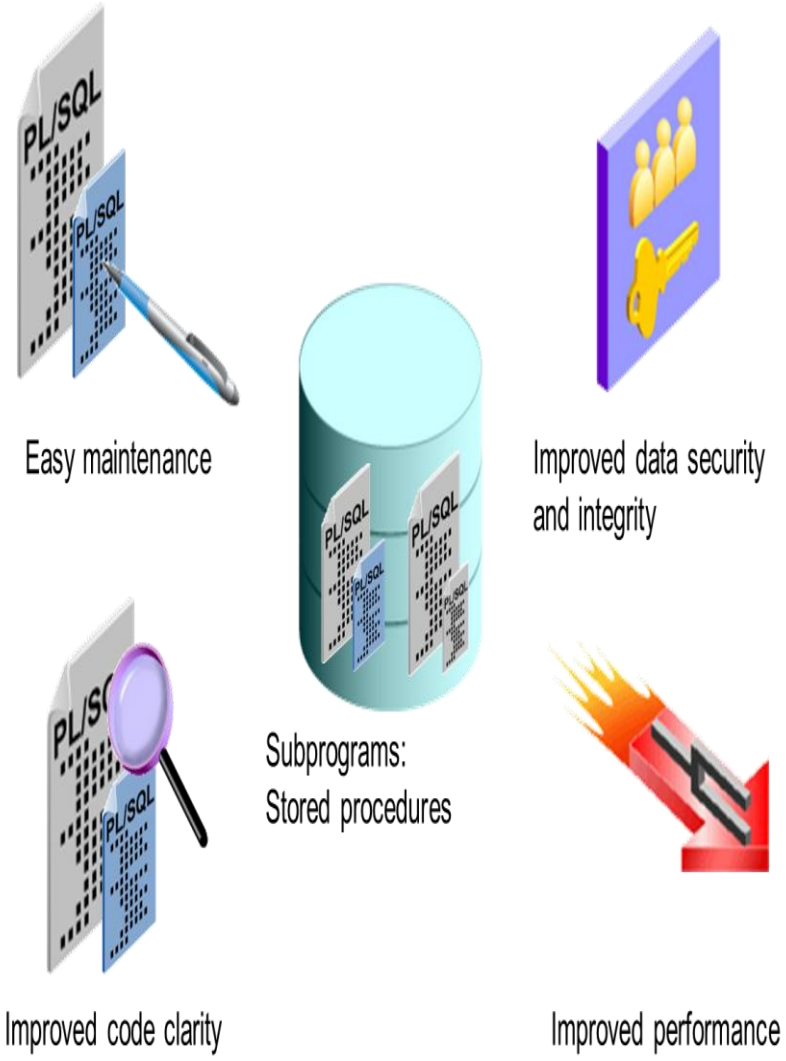
Utilidad

Los **procedimientos almacenados** SQL son importantes porque mejoran el rendimiento, la seguridad y la mantenibilidad de las bases de datos. Es decir al agrupar código SQL en un solo objeto, se reduce el tráfico de red, se facilita la reutilización del código y se mejora la seguridad al controlar el acceso a los datos.

Universidad Tecnológica del Perú

Procedimientos Almacenados

DEFINICIÓN

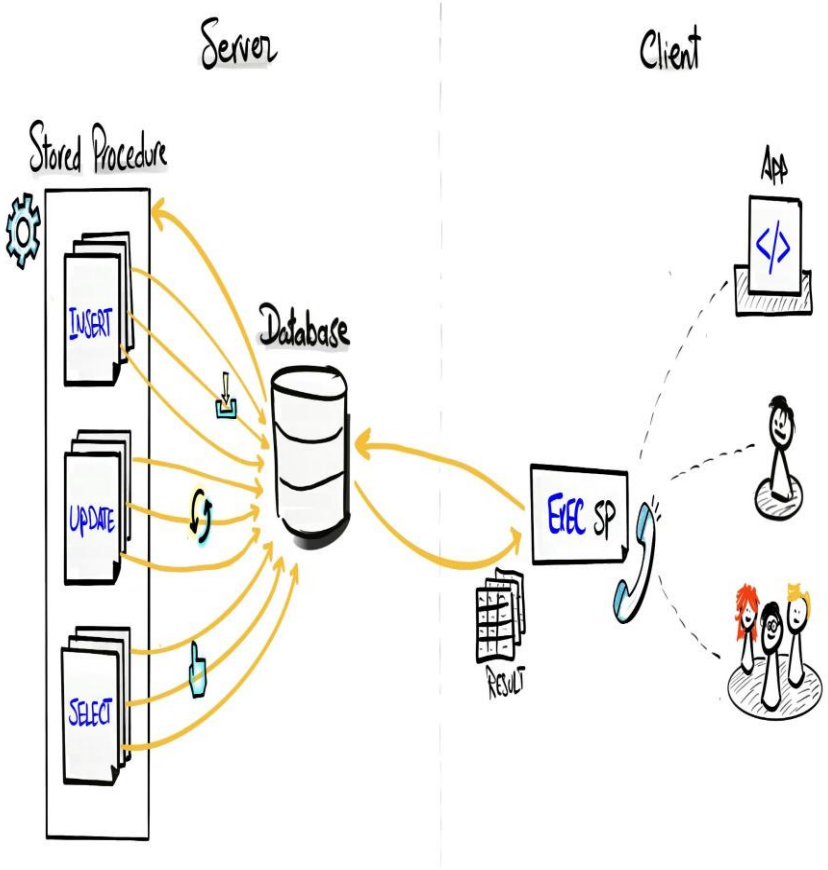


Un procedimiento almacenado es un conjunto de sentencias SQL que se almacenan en la base de datos y pueden ejecutarse varias veces, evitando la repetición del mismo código y mejorando la eficiencia y seguridad.

Es una especie de función encapsulada que permite ejecutar operaciones complejas, pasar parámetros, incluir lógica condicional y retornar resultados

Procedimientos Almacenados

Ventajas



Reutilización de código: se pueden ejecutar múltiples veces sin reescribir el SQL.

Mejora del rendimiento: se compilan una vez y se almacenan en caché.

Seguridad: se puede restringir el acceso a las tablas mediante permisos sobre el procedimiento.

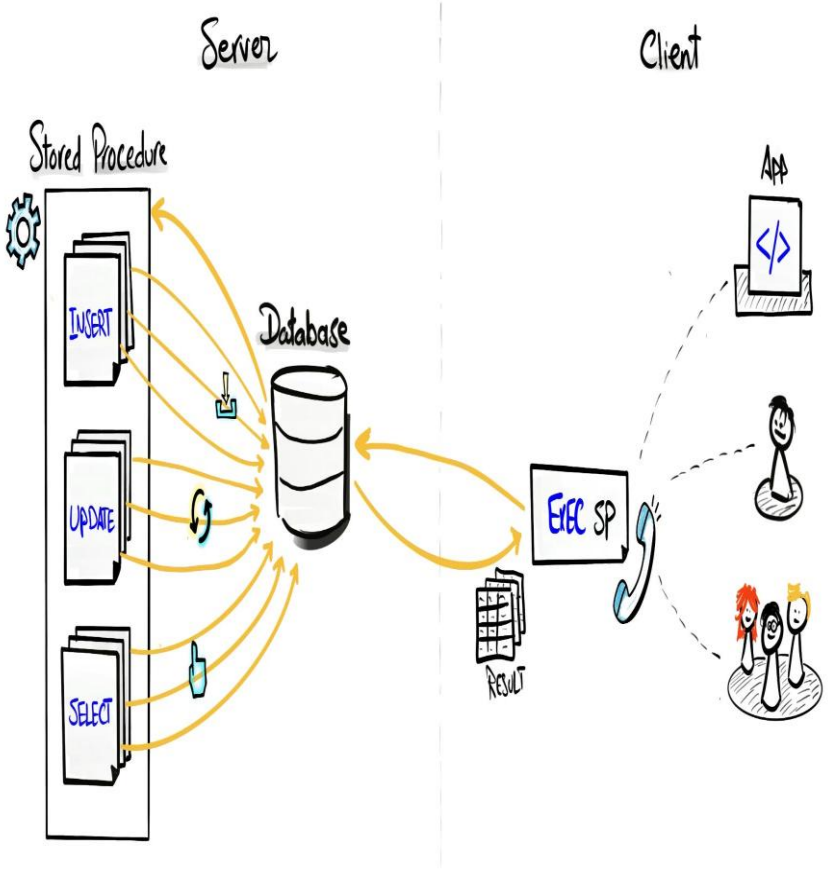
Mantenimiento: si cambia la lógica, solo se modifica en un lugar.

Control transaccional: permite definir el control de errores y el manejo de transacciones.

Procedimientos Almacenados

Universidad Tecnológica del Perú

Sintaxis Básica




```sql
CREATE PROCEDURE NombreProcedimiento
  -- Parámetros que empiezan con @
    @param1 TipoDato,
    @param2 TipoDato = ValorPorDefecto
AS
BEGIN
    -- Sentencias SQL
    -- Declaración y asignación de variables
    -- Código del procedimiento
END;
```

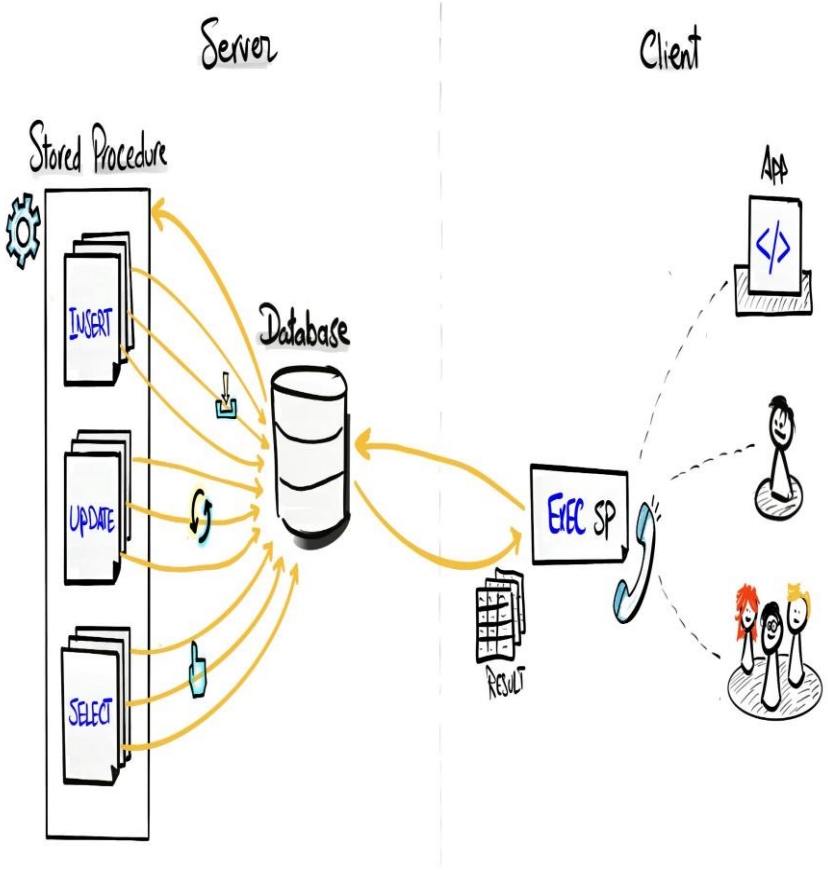
```sql
--Ejecución del Procedimiento
EXEC NombreProcedimiento @param1 = valor1, @param2 = valor2;
```

Desaprende lo que te limita

Procedimientos Almacenados

Universidad Tecnológica del Perú

Ejemplos

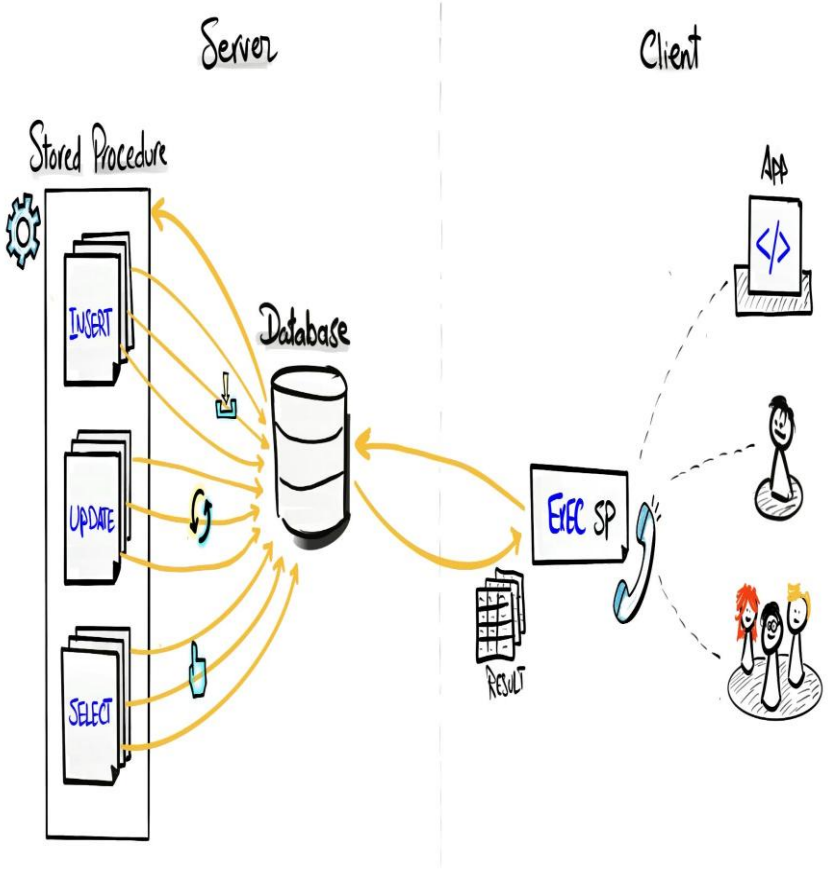



```
--Crear un procedimiento para mostrar los clientes
CREATE PROCEDURE mostCliente
AS
    SELECT * FROM Customers


--Ejecución del procedimiento mostCliente
EXEC mostCliente


--Crear un procedimiento para mostrar los clientes
CREATE PROCEDURE mostCliente
AS
BEGIN
    SELECT * FROM Customers
END;
```

Procedimientos Almacenados

Universidad Tecnológica del Perú

Ejemplos



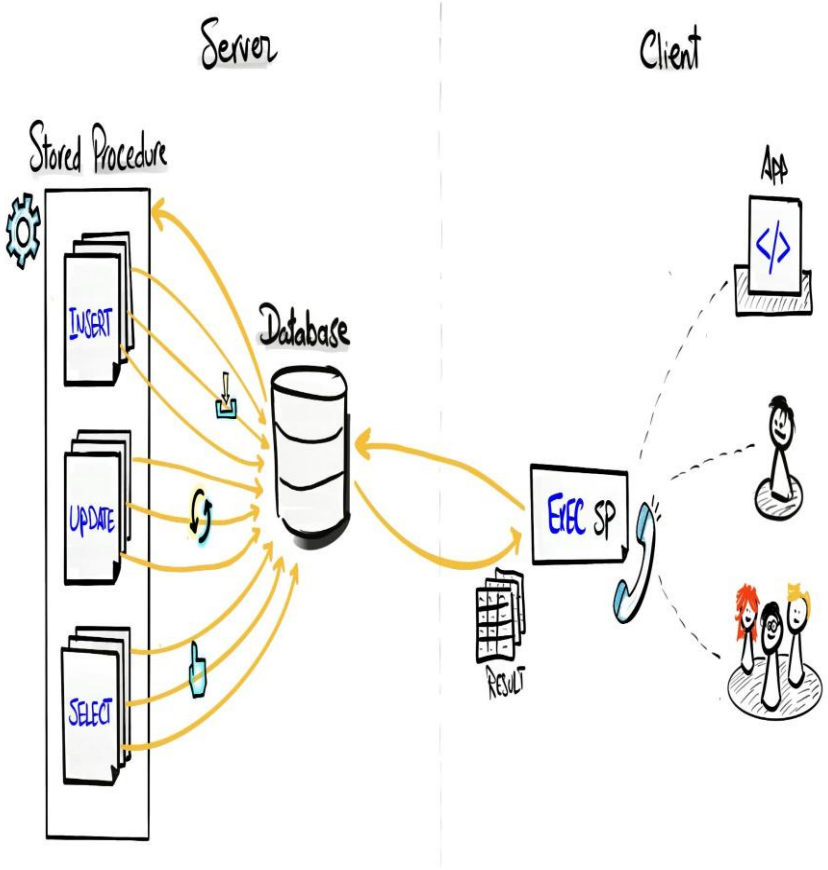

```sql
--Crear un procedimiento para obener los pedidos de un cliente
CREATE PROCEDURE ObtenerPedidosCliente
    @IDCliente NCHAR(5)
AS
BEGIN
    SELECT OrderID, OrderDate, ShipCountry
    FROM Orders
    WHERE CustomerID = @IDCliente;
END;
```

```sql
--Ejecución del procedimiento para el cliente ALFKI
EXEC ObtenerPedidosCliente @IDCliente = 'ALFKI';
```

Procedimientos Almacenados

Universidad Tecnológica del Perú

Sintaxis Básica



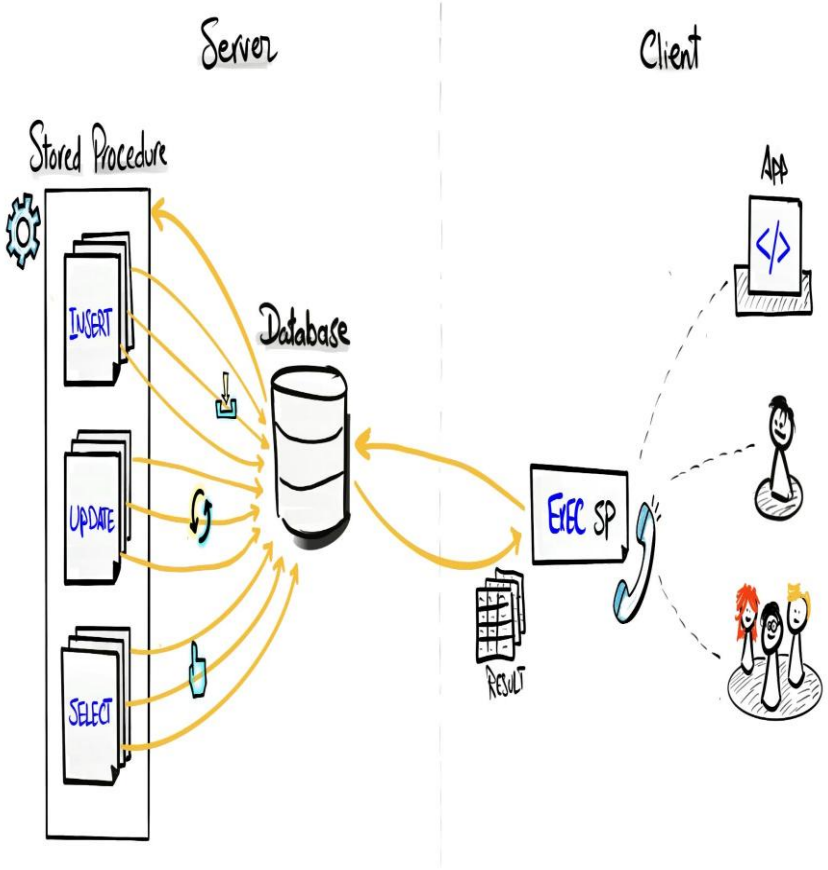
❖ Modificar un procedimiento

```
ALTER PROCEDURE NombreProcedimiento
-- Nueva definición...


--Modifica el procedimiento anterior
--ingresando un parametro más pais de envio
ALTER PROCEDURE ObtenerPedidosCliente
    @IDCliente NCHAR(5),
    @PaisEnvio NVARCHAR(40)
AS
BEGIN
    SELECT OrderID, OrderDate, ShipCountry
    FROM Orders
    WHERE CustomerID = @IDCliente
      AND ShipCountry = @PaisEnvio;
END;
```

Procedimientos Almacenados

Universidad Tecnológica del Perú

Sintaxis Básica



- **Eliminar un procedimiento**

```
DROP PROCEDURE NombreProcedimiento;
```

- **Ver procedimientos existentes**

```
SELECT * FROM sys.procedures;
```

- **Ver el procedimiento**

```
EXEC sp_helptext 'NombreProcedimiento';
```

Universidad Tecnológica del Perú

Ejemplos

```
EXEC BuscarProductoPorPrecio @minimo=50, @maximo=100
```

Results | Messages

	ProductName	UnitPrice
1	Mishi Kobe Niku	97.00
2	Carnarvon Tigers	62.50
3	Sir Rodney's Marmalade	81.00
4	Manjimup Dried Apples	53.00
5	Raclette Courdavault	55.00

❖ Procedimiento con lógica condicional

```
CREATE PROCEDURE BuscarProductoPorPrecio
    @Minimo DECIMAL(10,2),
    @Maximo DECIMAL(10,2)
AS
BEGIN
    IF @Minimo < 0 OR @Maximo < 0
        PRINT 'Valores inválidos'
    ELSE
        SELECT ProductName, UnitPrice
        FROM Products
        WHERE UnitPrice BETWEEN @Minimo AND @Maximo;
END;
```

Universidad Tecnológica del Perú

Ejemplos

```sql
DECLARE @Cantidad INT;
EXEC ObtenerTotalPedidosCliente @IDCliente = 'VINET',
                                @Total = @Cantidad OUTPUT;
PRINT @Cantidad;
```

Messages

5

❖ Procedimientos con salida (OUTPUT)

```sql
CREATE PROCEDURE ObtenerTotalPedidosCliente
    @IDCliente NCHAR(5),
    @Total INT OUTPUT
AS
BEGIN
    SELECT @Total = COUNT(*)
    FROM Orders
    WHERE CustomerID = @IDCliente;
END;
```

Procedimientos Almacenados

Universidad Tecnológica del Perú

Ejemplos

```sql
CREATE PROCEDURE ClientesConPedidosRecientes
AS
BEGIN
    SELECT DISTINCT c.CustomerID, c.CompanyName, o.OrderDate
    FROM Customers c
    INNER JOIN Orders o ON c.CustomerID = o.CustomerID
    WHERE o.OrderDate >= DATEADD(MONTH, -1, GETDATE())
    ORDER BY o.OrderDate DESC;
END;
```

- Crear un procedimiento almacenado llamado ClientesConPedidosRecientes que recupere una lista de clientes distintos junto con sus fechas de pedido, únicamente si han realizado al menos un pedido en el último mes. La información debe obtenerse a partir de la relación entre las tablas Customers y Orders, ordenando los resultados por fecha de pedido en orden descendente

Procedimientos Almacenados

Universidad Tecnológica del Perú

Ejemplos

```sql
CREATE PROCEDURE PedidosPorPais
    @Pais NVARCHAR(40)
AS
BEGIN
    SELECT OrderID, CustomerID, OrderDate, ShipCountry
    FROM Orders
    WHERE ShipCountry = @Pais
    ORDER BY OrderDate DESC;
END;
```

- Crear un procedimiento almacenado llamado PedidosPorPais que reciba como parámetro el nombre de un país y devuelva los pedidos realizados a ese país, mostrando el ID del pedido, el ID del cliente, la fecha del pedido y el país de envío, ordenados de forma descendente por la fecha del pedido. Este procedimiento permite filtrar dinámicamente los pedidos según el país de destino.

Procedimientos Almacenados

Universidad Tecnológica del Perú

Ejemplos

```sql
CREATE PROCEDURE ProductosPorProveedor
    @IDProveedor INT
AS
BEGIN
    SELECT ProductName, UnitPrice, UnitsInStock
    FROM Products
    WHERE SupplierID = @IDProveedor;
END;
```

- Diseñar un procedimiento almacenado llamado ProductosPorProveedor que reciba como parámetro el identificador de un proveedor (@IDProveedor) y retorne el nombre, precio unitario y unidades en stock de todos los productos suministrados por dicho proveedor registrados en la base de datos Northwind.

Procedimientos Almacenados

Universidad Tecnológica del Perú

Ejemplos

```sql
CREATE PROCEDURE TotalVentasPorEmpleado
    @IDEmpleado INT
AS
BEGIN
    SELECT e.FirstName + ' ' + e.LastName AS Empleado,
           SUM(o.Freight) AS TotalFlete
    FROM Orders o
    INNER JOIN Employees e ON o.EmployeeID = e.EmployeeID
    WHERE o.EmployeeID = @IDEmpleado
    GROUP BY e.FirstName, e.LastName;
END;
```

- Diseñe un procedimiento almacenado denominado TotalVentasPorEmpleado que reciba como parámetro el identificador de un empleado. El procedimiento debe retornar el nombre completo del empleado junto con el total acumulado del costo de flete (Freight) correspondiente a todos los pedidos que haya gestionado. El resultado debe agruparse por nombre del empleado.

Procedimientos Almacenados

Universidad Tecnológica del Perú

Ejemplos

```sql
CREATE PROCEDURE AjustaPrecUnit(
    @Ajuste decimal(10,4),
    @CodProd int
) AS
BEGIN
    DECLARE @Factor money;
    SET @Factor = 1.1;

    UPDATE Products
    SET UnitPrice = UnitPrice*@Ajuste*@Factor
    WHERE ProductID = @CodProd;

    UPDATE OrderDetails
    SET UnitPrice = UnitPrice*@Ajuste*@Factor
    WHERE ProductID = @CodProd;
END;
```

- Cree un store procedure que reciba un parámetro tipo precio unitario llamado ajuste y otro tipo código producto llamado codprod. Cree una constante factor con valor 1.1 y al precio unitario en las tablas producto y detalle de ordenes, multiplíquelos por el ajuste y el factor pero solo al producto cuyo código coincida con el parámetro codprod.

FUNCIONES

DEFINICIÓN

Una función definida por el usuario en SQL Server (UDF - User Defined Function) es una rutina programada por el usuario que acepta parámetros de entrada, realiza operaciones internas (consultas, cálculos, condiciones, etc.) y devuelve un valor (escalar o tabular).

FUNCIONES

Universidad Tecnológica del Perú

DEFINICIÓN

```
CREATE FUNCTION dbo.ObtenerDescuento (@precio DECIMAL(10,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN @precio * 0.10;
END;
```

```
SELECT dbo.ObtenerDescuento(100.25)
```

Results | Messages

	(No column name)
1	10.03

```
SELECT ProductName, UnitPrice, dbo.ObtenerDescuento(UnitPrice) AS Descuento
FROM Products;
```

Results | Messages

	ProductName	UnitPrice	Descuento
1	Chai	18.00	1.80
2	Chang	19.00	1.90
3	Aniseed Syrup	10.00	1.00
4	Chef Anton's Cajun Seasoning	22.00	2.20
5	Chef Anton's Gumbo Mix	21.35	2.14

Funciones escalares

Devuelven un solo valor escalar (INT, VARCHAR, DATE, etc.). Se usan dentro de consultas como si fueran una columna más.

Sintaxis:

```
CREATE FUNCTION NombreFuncion (@param1 tipo, @param2 tipo)
RETURNS tipo_resultado
AS
BEGIN
    DECLARE @resultado tipo_resultado;
    -- Lógica
    RETURN @resultado;
END;
```

FUNCIONES

Universidad Tecnológica del Perú

DEFINICIÓN

```
CREATE FUNCTION dbo.PedidosPorCliente (@IDCliente NCHAR(5))
RETURNS TABLE
AS
RETURN (
    SELECT OrderID, OrderDate, Freight
    FROM Orders
    WHERE CustomerID = @IDCliente
);
```

```
SELECT * FROM dbo.PedidosPorCliente('ALFKI');
```

Results | Messages

	OrderID	OrderDate	Freight
1	10643	1997-08-25 00:00:00.000	29.46
2	10692	1997-10-03 00:00:00.000	61.02
3	10702	1997-10-13 00:00:00.000	23.94
4	10835	1998-01-15 00:00:00.000	69.53
5	10952	1998-03-16 00:00:00.000	40.42
6	11011	1998-04-09 00:00:00.000	1.21

Funciones con valores de tabla en línea (Inline Table-Valued Functions**)**

Devuelven una tabla directamente, sin cuerpo BEGIN...END

Son muy eficientes y funcionan como vistas parametrizadas.

Sintaxis:

```
CREATE FUNCTION NombreFuncion (@param tipo)
RETURNS TABLE
AS
RETURN (
    SELECT columnas
    FROM tabla
    WHERE condición
);
```

Universidad Tecnológica del Perú

DEFINICIÓN

```sql
CREATE FUNCTION dbo.ClientesPorPais (@Pais NVARCHAR(50))
RETURNS @Resultado TABLE (
    CustomerID NCHAR(5),
    CompanyName NVARCHAR(40)
)
AS
BEGIN
    INSERT INTO @Resultado
    SELECT CustomerID, CompanyName
    FROM Customers
    WHERE Country = @Pais;
    RETURN;
END;

SELECT * FROM dbo.ClientesPorPais('Germany');
```

Results | Messages

	CustomerID	CompanyName
1	ALFKI	Alfreds Futterkiste
2	BLAUS	Blauer See Delikatessen
3	DRACD	Drachenblut Delikatessen
4	FRANK	Frankenversand
5	KOENE	Königlich Essen

Funciones con valores de tabla con múltiples instrucciones (Multistatement Table-Valued Functions**)**

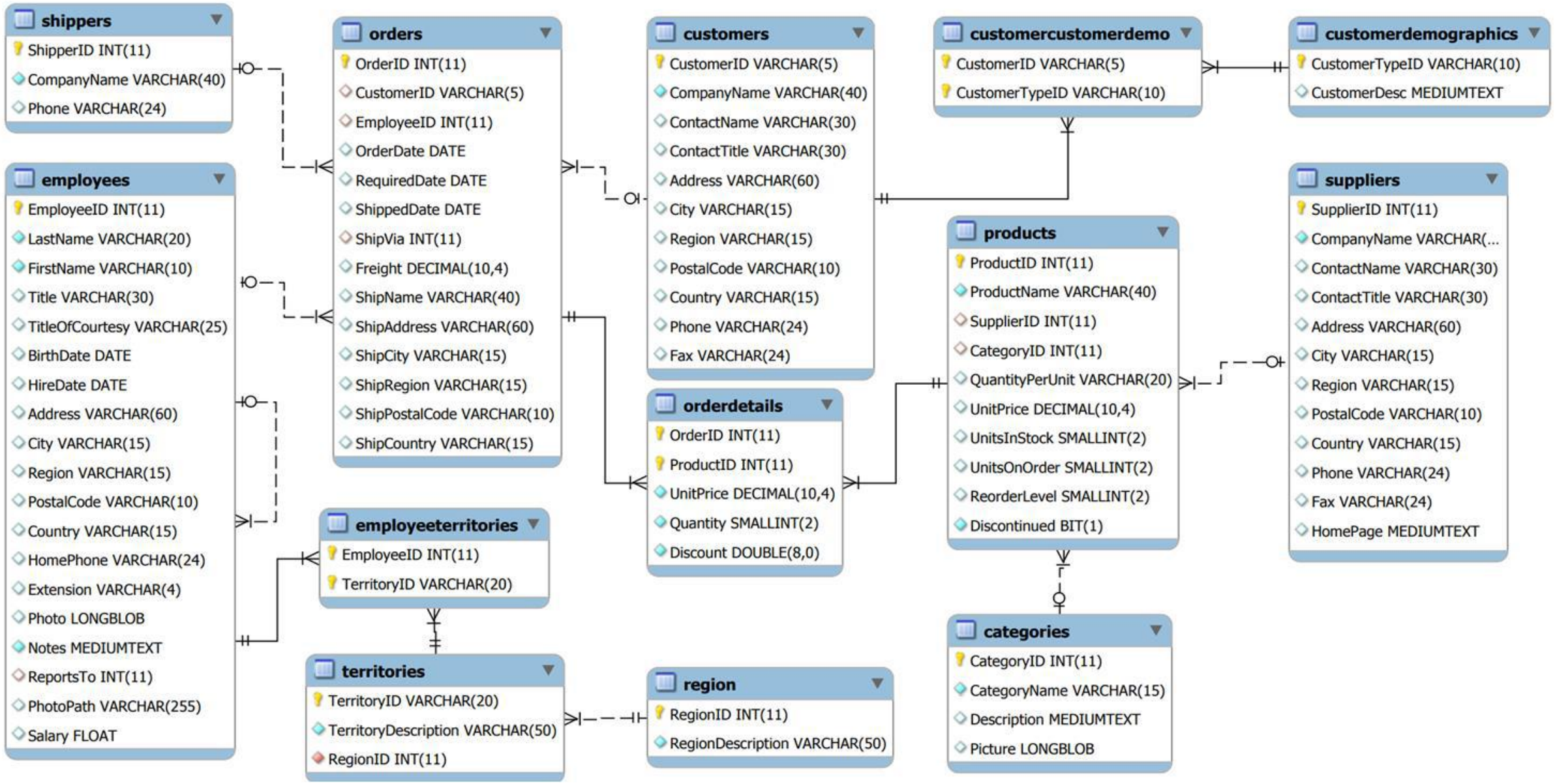
- Devuelven una tabla, pero permiten instrucciones más complejas, condicionales, variables, etc.

Sintaxis:

```sql
CREATE FUNCTION NombreFuncion (@param tipo)
RETURNS @TablaRetorno TABLE (col1 tipo, col2 tipo, ...)
AS
BEGIN
    -- Lógica con múltiples instrucciones
    INSERT INTO @TablaRetorno
    SELECT ...
    RETURN;
END;
```

EJERCICIOS

Universidad Tecnológica del Perú



Cierre

Cual es la finalidad de un procedimiento almacenado.

Que diferencia existe entre el procedimiento almacenado y función

Universidad Tecnológica del Perú